

选做部分：实现上述三个测站的组网解算，并实现批量计算和精度评定。

代码：

```
import numpy as np
from datetime import datetime
from matplotlib import pyplot as plt

def read_data_from_file(file_path):
    data = []
    with open(file_path, 'r', encoding='utf-8') as file:
        lines = file.readlines()
        for line in lines[1:]:
            parts = line.strip().split(',')
            if len(parts) != 6:
                continue
            try:
                entry = {
                    'time': datetime.fromisoformat(parts[0].strip()),
                    'satellite': parts[1].strip(),
                    'pseudo_range': float(parts[2].strip()),
                    'carrier_phase': float(parts[3].strip()),
                    'doppler': float(parts[4].strip()),
                    'signal_strength': float(parts[5].strip())
                }
            except ValueError as e:
                print(f"Error parsing line: {line}. Error: {e}")
                continue
            data.append(entry)
    return data

def parse_observations(data):
    obs_dict = {}
    for entry in data:
        time = entry['time']
        if time not in obs_dict:
            obs_dict[time] = {}
        obs_dict[time][entry['satellite']] = entry
    return obs_dict

def calculate_differences(base_obs, rover_obs):
```

```

time_keys = set(base_obs.keys()) & set(rover_obs.keys())
dd_pseudo_ranges = {}
dd_carrier_phases = {}
for time in time_keys:
    base_data = base_obs[time]
    rover_data = rover_obs[time]
    common_sats = set(base_data.keys()) & set(rover_data.keys())
    for sat in common_sats:
        base_pr = base_data[sat]['pseudo_range']
        base_cp = base_data[sat]['carrier_phase']
        rover_pr = rover_data[sat]['pseudo_range']
        rover_cp = rover_data[sat]['carrier_phase']
        if sat not in dd_pseudo_ranges:
            dd_pseudo_ranges[sat] = []
            dd_carrier_phases[sat] = []
        dd_pseudo_ranges[sat].append(rover_pr - base_pr)
        dd_carrier_phases[sat].append(rover_cp - base_cp)
return dd_pseudo_ranges, dd_carrier_phases

def calculate_relative_position(base_coords, dd_pseudo_ranges,
dd_carrier_phases):
    base_X, base_Y, base_Z = base_coords
    rover_positions = {}
    for sat in dd_pseudo_ranges:
        A = np.ones((len(dd_pseudo_ranges[sat]), 4))
        A[:, 1:] = np.random.randn(len(dd_pseudo_ranges[sat]), 3)
        b = np.array(dd_pseudo_ranges[sat])
        x, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
        relative_position = x[1:]
        rover_position = np.array([base_X, base_Y, base_Z]) +
relative_position
        rover_positions[sat] = rover_position
    return rover_positions

def read0(ephemerisfile, outputfile):
    with open(ephemerisfile, 'r', encoding='utf-8') as fide:
        lines = fide.readlines()
    data = {}
    observations = []
    header_end = 0

    # 读取基准站坐标

```

```
for content in lines:
    if "APPROX POSITION XYZ" in content:
        parts = content.split()
        data['nx'] = float(parts[0])
        data['ny'] = float(parts[1])
        data['nz'] = float(parts[2])
        break

# 读取天线偏移信息
for content in lines:
    if "ANTENNA: DELTA H/E/N" in content:
        parts = content.split()
        data['delta_H'] = float(parts[0])
        data['delta_E'] = float(parts[1])
        data['delta_N'] = float(parts[2])
        break

# 读取观测时间间隔
for content in lines:
    if "INTERVAL" in content:
        data['interval'] = float(content.split()[0])
        break

# 读取观测历元时间
for content in lines:
    if "TIME OF FIRST OBS" in content:
        parts = content.split()
        data['first_obs'] = {
            'year': int(parts[0]),
            'month': int(parts[1]),
            'day': int(parts[2]),
            'hour': int(parts[3]),
            'minute': int(parts[4]),
            'second': float(parts[5])
        }
    elif "TIME OF LAST OBS" in content:
        parts = content.split()
        data['last_obs'] = {
            'year': int(parts[0]),
            'month': int(parts[1]),
            'day': int(parts[2]),
            'hour': int(parts[3]),
            'minute': int(parts[4]),
            'second': float(parts[5])
        }
```

```

    }

# 读取 END OF HEADER 位置
for i, content in enumerate(lines):
    if "END OF HEADER" in content:
        header_end = i
        break

# 读取观测值
i = header_end + 1
while i < len(lines):
    line = lines[i].strip()
    if not line: # 跳过空行
        i += 1
        continue

    if line.startswith('>'):
        parts = line.split()
        observation_epoch = {
            'year': int(parts[1]),
            'month': int(parts[2]),
            'day': int(parts[3]),
            'hour': int(parts[4]),
            'minute': int(parts[5]),
            'second': float(parts[6]),
            'satellite_count': int(parts[8]),
            'observations': []
        }
        i += 1

# 读取该历元下的所有卫星观测值
sat_count = 0
while i < len(lines) and sat_count < observation_epoch['satellite_count']:
    obs_line = lines[i].strip()
    if not obs_line or obs_line.startswith('>'):
        break # 遇到空行或下一个历元标记则停止

    obs_parts = obs_line.split()
    if len(obs_parts) >= 5: # 确保有足够的字段
        try:
            observation_epoch['observations'].append({
                'satellite': obs_parts[0],
                'pseudo_range': float(obs_parts[1]),

```

```

        'carrier_phase': float(obs_parts[2]),
        'doppler': float(obs_parts[3]),
        'signal_strength': float(obs_parts[4])
    })
    sat_count += 1
except ValueError as e:
    print(f"解析观测数据时出错 (行 {i}): {e}")
else:
    print(f" (行 {i}): {obs_line}")

i += 1

observations.append(observation_epoch)
else:
    i += 1

with open(outputfile, 'w', encoding='utf-8') as fidu:

fidu.write("time,satellite,pseudo_range,carrier_phase,doppler,signal_
strength\n")
    for obs in observations:
        time_str = f"{obs['year']:04d}-{obs['month']:02d}-
{obs['day']:02d}T{obs['hour']:02d}:{obs['minute']:02d}:{obs['second']
:06.3f}"
        for o in obs['observations']:
            fidu.write(

f"{time_str},{o['satellite']},{o['pseudo_range']},{o['carrier_phase']
},{o['doppler']},{o['signal_strength']}\n")

    return data, observations

def calculate_dop(sat_positions, receiver_position):
    """计算DOP值(精度衰减因子)"""
    # 卫星位置矩阵
    sat_matrix = np.array(list(sat_positions.values()))

    # 接收机位置
    rx_X, rx_Y, rx_Z = receiver_position

    # 构建设计矩阵A
    A = np.zeros((len(sat_positions), 4))
    for i, (sat_id, (sat_X, sat_Y, sat_Z)) in

```

```

enumerate(sat_positions.items()):
    # 计算卫星到接收机的距离
    rho = np.sqrt((sat_X - rx_X) ** 2 + (sat_Y - rx_Y) ** 2 +
(sat_Z - rx_Z) ** 2)

    # 设计矩阵元素
    A[i, 0] = (sat_X - rx_X) / rho
    A[i, 1] = (sat_Y - rx_Y) / rho
    A[i, 2] = (sat_Z - rx_Z) / rho
    A[i, 3] = 1.0 # 钟差参数

# 计算信息矩阵 Q
Q = np.linalg.inv(np.dot(A.T, A))

# 计算各种 DOP 值
gdop = np.sqrt(np.trace(Q))
pdop = np.sqrt(Q[0, 0] + Q[1, 1] + Q[2, 2])
hdop = np.sqrt(Q[0, 0] + Q[1, 1])
vdop = np.sqrt(Q[2, 2])

return {
    'GDOP': gdop,
    'PDOP': pdop,
    'HDOP': hdop,
    'VDOP': vdop
}

def visualize_dop_values(dop_results, title):
    """可视化 DOP 值"""
    dop_types = list(dop_results.keys())
    values = list(dop_results.values())

    plt.figure(figsize=(10, 6))
    bars = plt.bar(dop_types, values, color='skyblue')

    # 添加数值标签
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width() / 2., height,
            f'{height:.2f}',
            ha='center', va='bottom')

    plt.title(title)

```

```

plt.ylabel('DOP 值')
plt.ylim(0, max(values) * 1.2) # 设置 y 轴范围, 留出一些空间
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
return plt

def visualize_baseline_and_sats(base_coords, rover_positions,
sat_positions, title):
    """可视化基线和卫星分布"""
    fig = plt.figure(figsize=(12, 10))

    # 3D 图: 卫星和接收机位置
    ax1 = fig.add_subplot(221, projection='3d')

    # 绘制基准站
    base_X, base_Y, base_Z = base_coords
    ax1.scatter(base_X, base_Y, base_Z, c='red', s=100, marker='^',
label='基准站')

    # 绘制流动站
    colors = ['blue', 'green', 'purple']
    labels = ['流动站 1', '流动站 2', '流动站 3']
    for i, (sat_id, pos) in enumerate(rover_positions.items()):
        if i < len(colors):
            ax1.scatter(pos[0], pos[1], pos[2], c=colors[i], s=100,
marker='o', label=labels[i])

    # 绘制卫星
    sat_X = [pos[0] for pos in sat_positions.values()]
    sat_Y = [pos[1] for pos in sat_positions.values()]
    sat_Z = [pos[2] for pos in sat_positions.values()]
    ax1.scatter(sat_X, sat_Y, sat_Z, c='gray', s=50, marker='*',
label='卫星')

    # 绘制基线
    for i, (sat_id, pos) in enumerate(rover_positions.items()):
        if i < len(colors):
            ax1.plot([base_X, pos[0]], [base_Y, pos[1]], [base_Z,
pos[2]], c=colors[i], linestyle='-', alpha=0.6)

    ax1.set_xlabel('X (m)')
    ax1.set_ylabel('Y (m)')

```

```

ax1.set_zlabel('Z (m)')
ax1.set_title('基线和卫星分布')
ax1.legend()

# 2D 图: 水平面上的分布
ax2 = fig.add_subplot(222)
ax2.scatter(base_X, base_Y, c='red', s=100, marker='^', label='基
准站')

for i, (sat_id, pos) in enumerate(rover_positions.items()):
    if i < len(colors):
        ax2.scatter(pos[0], pos[1], c=colors[i], s=100, marker='o',
label=labels[i])
        ax2.plot([base_X, pos[0]], [base_Y, pos[1]], c=colors[i],
linestyle='--', alpha=0.6)

ax2.set_xlabel('X (m)')
ax2.set_ylabel('Y (m)')
ax2.set_title('水平面上的基线分布')
ax2.legend()
ax2.grid(True)

# 计算并可视化 DOP 值
dop_results = calculate_dop(sat_positions, base_coords)
ax3 = fig.add_subplot(212)

dop_types = ['GDOP', 'PDOP', 'HDOP', 'VDOP']
values = [dop_results[key] for key in dop_types]

bars = ax3.bar(dop_types, values, color='lightgreen')

for bar in bars:
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width() / 2., height,
            f'{height:.2f}',
            ha='center', va='bottom')

ax3.set_title('精度衰减因子 (DOP)')
ax3.set_ylabel('DOP 值')
ax3.set_ylim(0, max(values) * 1.2)
ax3.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.suptitle(title, fontsize=16)

```

```

plt.subplots_adjust(top=0.9)

return plt

if __name__ == "__main__":
    # 处理新建文本文档（4）
    base_coords = (-2623864.5572, 3976998.0927, 4226091.6081)
    data4, observations4 = readO('4.txt', 'station4.txt')
    data5, observations5 = readO('5.txt', 'station5.txt')
    data6, observations6 = readO('6.txt', 'station6.txt')

    station4_data = read_data_from_file('station4.txt')
    station5_data = read_data_from_file('station5.txt')
    station6_data = read_data_from_file('station6.txt')

    station4_obs = parse_observations(station4_data)
    station5_obs = parse_observations(station5_data)
    station6_obs = parse_observations(station6_data)

    # 计算新建文本文档（4）与（5）的双差
    dd_pseudo_ranges_4_5, dd_carrier_phases_4_5 =
calculate_differences(station4_obs, station5_obs)
    rover_positions_4_5 = calculate_relative_position(base_coords,
dd_pseudo_ranges_4_5, dd_carrier_phases_4_5)

    # 计算新建文本文档（4）与（6）的双差
    dd_pseudo_ranges_4_6, dd_carrier_phases_4_6 =
calculate_differences(station4_obs, station6_obs)
    rover_positions_4_6 = calculate_relative_position(base_coords,
dd_pseudo_ranges_4_6, dd_carrier_phases_4_6)

    # 计算新建文本文档（5）与（6）的双差
    dd_pseudo_ranges_5_6, dd_carrier_phases_5_6 =
calculate_differences(station5_obs, station6_obs)
    rover_positions_5_6 = calculate_relative_position(base_coords,
dd_pseudo_ranges_5_6, dd_carrier_phases_5_6)

    print("新建文本文档（4）与（5）的基线解算结果：")
    for sat, pos in rover_positions_4_5.items():
        print(f"Satellite {sat} Rover Position (X, Y, Z):
{pos[0]:.6f}, {pos[1]:.6f}, {pos[2]:.6f}")

    print("\n新建文本文档（4）与（6）的基线解算结果：")
    for sat, pos in rover_positions_4_6.items():

```

```

        print(f"Satellite {sat} Rover Position (X, Y, Z):
{pos[0]:.6f}, {pos[1]:.6f}, {pos[2]:.6f}")

    print("\n新建文本文档（5）与（6）的基线解算结果: ")
    for sat, pos in rover_positions_5_6.items():
        print(f"Satellite {sat} Rover Position (X, Y, Z):
{pos[0]:.6f}, {pos[1]:.6f}, {pos[2]:.6f}")

sat_positions = {}
for sat_id in ['G01', 'G02', 'G03', 'G04', 'G05', 'G06', 'G07',
'G08']:
    # 生成随机卫星位置（实际应用中应替换为真实卫星位置）
    sat_X = base_coords[0] + np.random.uniform(2000000, 2500000)
    sat_Y = base_coords[1] + np.random.uniform(2000000, 2500000)
    sat_Z = base_coords[2] + np.random.uniform(2000000, 2500000)
    sat_positions[sat_id] = (sat_X, sat_Y, sat_Z)

# 计算并可视化 4-5 基线的 DOP 值
dop_4_5 = calculate_dop(sat_positions, base_coords)
plt_4_5 = visualize_dop_values(dop_4_5, '新建文本文档（4）与（5）的 DOP 值
')
plt_4_5.savefig('dop_4_5.png')

# 计算并可视化 4-6 基线的 DOP 值
dop_4_6 = calculate_dop(sat_positions, base_coords)
plt_4_6 = visualize_dop_values(dop_4_6, '新建文本文档（4）与（6）的 DOP 值
')
plt_4_6.savefig('dop_4_6.png')

# 计算并可视化 5-6 基线的 DOP 值
dop_5_6 = calculate_dop(sat_positions, base_coords)
plt_5_6 = visualize_dop_values(dop_5_6, '新建文本文档（5）与（6）的 DOP 值
')
plt_5_6.savefig('dop_5_6.png')

# 可视化基线和卫星分布
plt_baseline_4_5 = visualize_baseline_and_sats(base_coords,
rover_positions_4_5, sat_positions,
'新建文本文档（4）与（5）的基线解算
结果')
plt_baseline_4_5.savefig('baseline_4_5.png')

plt_baseline_4_6 = visualize_baseline_and_sats(base_coords,
rover_positions_4_6, sat_positions,

```

```

'新建文本文档（4）与（6）的基线解算
结果')
plt_baseline_4_6.savefig('baseline_4_6.png')

plt_baseline_5_6 = visualize_baseline_and_sats(base_coords,
rover_positions_5_6, sat_positions,

'新建文本文档（5）与（6）的基线解算
结果')
plt_baseline_5_6.savefig('baseline_5_6.png')

# 显示所有图表
plt.show()

```

结果：

```

D:\python\python.exe D:\python\选做部分\读取.py
(行 47): C20 24384755.883 126977834.292 43.000
(行 53): C30 21905838.109 114069444.839 49.000
(行 54): C29 22397330.711 116628776.641 47.700
(行 55): C27 25345009.117 131978102.641 41.100
(行 80): C20 24381063.734 126958607.896 43.800
(行 86): C30 21906080.633 114070705.880 49.100
(行 87): C29 22395114.930 116617240.922 47.600
(行 88): C27 25346756.922 131987205.045 40.000
(行 113): C20 24377373.617 126939392.652 43.200
(行 119): C30 21906324.734 114071979.113 48.800
(行 120): C29 22392901.391 116605715.419 46.600
(行 121): C27 25348506.336 131996315.684 40.300
(行 146): C20 24373685.953 126920188.526 44.200
(行 152): C30 21906571.367 114073264.588 49.500
(行 153): C29 22390690.188 116594200.197 46.500
(行 154): C27 25350257.906 132005434.527 40.200
(行 179): C20 24370000.258 126900996.221 44.300
(行 185): C30 21906821.141 114074562.889 48.900
(行 186): C29 22388481.289 116582695.828 47.000
(行 187): C27 25352010.484 132014562.210 40.000
(行 212): C20 24366316.734 126881814.734 42.200
(行 218): C30 21907072.703 114075873.062 48.900
(行 219): C29 22386273.781 116571201.372 47.000
(行 220): C27 25353764.156 132023697.742 38.900
(行 245): C20 24362635.172 126862645.216 42.500
(行 251): C30 21907326.563 114077196.244 48.100
(行 252): C29 22384069.250 116559717.984 48.100

```

```
python 版本控制
base_station.txt 基站.py station.txt baseline_4.5.png baseline_4.6.png dop_4.6.png dop_4.5.png
share
src
源代码
第三大作业
数据部分
1.txt
2.txt
3.txt
4.txt
5.txt
6.txt
333300CHN_R_202510003
333300CHN_R_202510003
444400CHN_R_202510007
444400CHN_R_202510007
555500CHN_R_202510007
555500CHN_R_202510007
baseline_4.5.png
baseline_4.6.png
baseline_5.6.png
dop_4.5.png
dop_4.6.png
dop_5.6.png
station.txt
station.txt
基站.py
333300CHN_R_202510003
333300CHN_R_202510007
555500CHN_R_202510007
LICENSE.txt
NEWS.txt
python.exe
1 time,satellite,pseudo_range,carrier_phase,doppler,signal_strength
2 2025-04-18T06:33:30.000,611,23444944.063,23444953.211,12330914.637,96885057.779
3 2025-04-18T06:33:30.000,617,28621716.75,28621722.957,108367848.235,84422592.274
4 2025-04-18T06:33:30.000,622,21902216.758,21902224.87,115804813.771,69077797.794
5 2025-04-18T06:33:30.000,609,2235925.195,2235924.566,116463564.832,90880537.346
6 2025-04-18T06:33:30.000,606,21087237.133,21087244.637,110814179.213,86348752.148
7 2025-04-18T06:33:30.000,619,1984587.602,1984591.113,104294519.956,8126471.316
8 2025-04-18T06:33:30.000,604,23532356.109,23532348.418,123663269.128,96361863.949
9 2025-04-18T06:33:30.000,614,22771472.039,22771482.777,119648687.433,92543384.071
10 2025-04-18T06:33:30.000,612,24224837.445,24224847.328,127280914.729,9939107.219
11 2025-04-18T06:33:30.000,609,23847253.773,23847269.953,125318282.949,97655888.826
12 2025-04-18T06:33:30.000,601,20954401.148,20954406.922,112014524.088,112014339.604
13 2025-04-18T06:33:30.000,602,21869913.367,21869921.438,21869921.438,21869921.355
14 2025-04-18T06:33:30.000,618,21683956.477,21683955.637,21683956.477,21683956.25
15 2025-04-18T06:33:30.000,615,2331429.306,2331429.891,21864839.832,21864839.844
16 2025-04-18T06:33:30.000,616,21473256.5,21473255.235,21473262.586,21473262.727
17 2025-04-18T06:33:30.000,617,19678444.396,19678444.406,19678444.396,19678444.396
18 2025-04-18T06:33:30.000,624,21563953.391,21563951.75,21563966.344,21563959.586
19 2025-04-18T06:33:30.000,613,26174791.805,26174802.137,26174799.934,26174802.621
20 2025-04-18T06:33:30.000,615,23483862.75,23483871.816,23483869.694,23483871.504
21 2025-04-18T06:33:30.000,621,26432398.532,26432398.934,26432398.532,26432398.532
22 2025-04-18T06:33:30.000,627,24194262.195,24194270.289,24194268.133,24194270.429
23 2025-04-18T06:33:30.000,630,25912772.523,25912783.785,25912781.77,25912783.895
24 2025-04-18T06:33:30.000,605,25488274.055,25488287.641,25488284.469,25488286.777
25 2025-04-18T06:33:30.000,613,38985382.773,38985312.443,25919121.391,156455528.146
26 2025-04-18T06:33:30.000,604,25023315.549,25023315.549,25023315.549,25023315.549
27 2025-04-18T06:33:30.000,603,37480154.182,37480154.551,18478452.159,150810583.123
28 2025-04-18T06:33:30.000,602,38895849.508,38895845.542,202489494.391,156577554.786
29 2025-04-18T06:33:30.000,601,37968455.211,37968456.809,197670850.346,152851024.181
30 2025-04-18T06:33:35.000,611,23461792.648,23461801.898,123292556.888,96072155.663
31 2025-04-18T06:33:35.000,617,28621716.75,28621722.957,108367848.235,84422592.274
32 2025-04-18T06:33:35.000,622,21902216.758,21902224.87,115804813.771,69077797.794
33 2025-04-18T06:33:35.000,609,2235925.195,2235924.566,116463564.832,90880537.346
34 2025-04-18T06:33:35.000,606,21087237.133,21087244.637,110814179.213,86348752.148
35 2025-04-18T06:33:35.000,619,1984587.602,1984591.113,104294519.956,8126471.316
36 2025-04-18T06:33:35.000,604,23532356.109,23532348.418,123663269.128,96361863.949
37 2025-04-18T06:33:35.000,614,22771472.039,22771482.777,119648687.433,92543384.071
38 2025-04-18T06:33:35.000,612,24224837.445,24224847.328,127280914.729,9939107.219
39 2025-04-18T06:33:35.000,609,23847253.773,23847269.953,125318282.949,97655888.826
40 2025-04-18T06:33:35.000,601,20954401.148,20954406.922,112014524.088,112014339.604
41 2025-04-18T06:33:35.000,602,21869913.367,21869921.438,21869921.438,21869921.355
42 2025-04-18T06:33:35.000,618,21683956.477,21683955.637,21683956.477,21683956.25
43 2025-04-18T06:33:35.000,615,2331429.306,2331429.891,21864839.832,21864839.844
44 2025-04-18T06:33:35.000,616,21473256.5,21473255.235,21473262.586,21473262.727
45 2025-04-18T06:33:35.000,617,19678444.396,19678444.406,19678444.396,19678444.396
46 2025-04-18T06:33:35.000,624,21563953.391,21563951.75,21563966.344,21563959.586
47 2025-04-18T06:33:35.000,613,26174791.805,26174802.137,26174799.934,26174802.621
48 2025-04-18T06:33:35.000,615,23483862.75,23483871.816,23483869.694,23483871.504
49 2025-04-18T06:33:35.000,621,26432398.532,26432398.934,26432398.532,26432398.532
50 2025-04-18T06:33:35.000,627,24194262.195,24194270.289,24194268.133,24194270.429
51 2025-04-18T06:33:35.000,630,25912772.523,25912783.785,25912781.77,25912783.895
52 2025-04-18T06:33:35.000,605,25488274.055,25488287.641,25488284.469,25488286.777
53 2025-04-18T06:33:35.000,613,38985382.773,38985312.443,25919121.391,156455528.146
54 2025-04-18T06:33:35.000,604,25023315.549,25023315.549,25023315.549,25023315.549
55 2025-04-18T06:33:35.000,603,37480154.182,37480154.551,18478452.159,150810583.123
56 2025-04-18T06:33:35.000,602,38895849.508,38895845.542,202489494.391,156577554.786
57 2025-04-18T06:33:35.000,601,37968455.211,37968456.809,197670850.346,152851024.181
58 2025-04-18T06:33:35.000,611,23461792.648,23461801.898,123292556.888,96072155.663
59 2025-04-18T06:33:35.000,617,28621716.75,28621722.957,108367848.235,84422592.274
60 2025-04-18T06:33:35.000,622,21902216.758,21902224.87,115804813.771,69077797.794
61 2025-04-18T06:33:35.000,609,2235925.195,2235924.566,116463564.832,90880537.346
62 2025-04-18T06:33:35.000,606,21087237.133,21087244.637,110814179.213,86348752.148
63 2025-04-18T06:33:35.000,619,1984587.602,1984591.113,104294519.956,8126471.316
64 2025-04-18T06:33:35.000,604,23532356.109,23532348.418,123663269.128,96361863.949
65 2025-04-18T06:33:35.000,614,22771472.039,22771482.777,119648687.433,92543384.071
66 2025-04-18T06:33:35.000,612,24224837.445,24224847.328,127280914.729,9939107.219
67 2025-04-18T06:33:35.000,609,23847253.773,23847269.953,125318282.949,97655888.826
68 2025-04-18T06:33:35.000,601,20954401.148,20954406.922,112014524.088,112014339.604
69 2025-04-18T06:33:35.000,602,21869913.367,21869921.438,21869921.438,21869921.355
70 2025-04-18T06:33:35.000,618,21683956.477,21683955.637,21683956.477,21683956.25
71 2025-04-18T06:33:35.000,615,2331429.306,2331429.891,21864839.832,21864839.844
72 2025-04-18T06:33:35.000,616,21473256.5,21473255.235,21473262.586,21473262.727
73 2025-04-18T06:33:35.000,617,19678444.396,19678444.406,19678444.396,19678444.396
74 2025-04-18T06:33:35.000,624,21563953.391,21563951.75,21563966.344,21563959.586
75 2025-04-18T06:33:35.000,613,26174791.805,26174802.137,26174799.934,26174802.621
76 2025-04-18T06:33:35.000,615,23483862.75,23483871.816,23483869.694,23483871.504
77 2025-04-18T06:33:35.000,621,26432398.532,26432398.934,26432398.532,26432398.532
78 2025-04-18T06:33:35.000,627,24194262.195,24194270.289,24194268.133,24194270.429
79 2025-04-18T06:33:35.000,630,25912772.523,25912783.785,25912781.77,25912783.895
80 2025-04-18T06:33:35.000,605,25488274.055,25488287.641,25488284.469,25488286.777
81 2025-04-18T06:33:35.000,613,38985382.773,38985312.443,25919121.391,156455528.146
82 2025-04-18T06:33:35.000,604,25023315.549,25023315.549,25023315.549,25023315.549
83 2025-04-18T06:33:35.000,603,37480154.182,37480154.551,18478452.159,150810583.123
84 2025-04-18T06:33:35.000,602,38895849.508,38895845.542,202489494.391,156577554.786
85 2025-04-18T06:33:35.000,601,37968455.211,37968456.809,197670850.346,152851024.181
86 2025-04-18T06:33:35.000,611,23461792.648,23461801.898,123292556.888,96072155.663
87 2025-04-18T06:33:35.000,617,28621716.75,28621722.957,108367848.235,84422592.274
88 2025-04-18T06:33:35.000,622,21902216.758,21902224.87,115804813.771,69077797.794
89 2025-04-18T06:33:35.000,609,2235925.195,2235924.566,116463564.832,90880537.346
90 2025-04-18T06:33:35.000,606,21087237.133,21087244.637,110814179.213,86348752.148
91 2025-04-18T06:33:35.000,619,1984587.602,1984591.113,104294519.956,8126471.316
92 2025-04-18T06:33:35.000,604,23532356.109,23532348.418,123663269.128,96361863.949
93 2025-04-18T06:33:35.000,614,22771472.039,22771482.777,119648687.433,92543384.071
94 2025-04-18T06:33:35.000,612,24224837.445,24224847.328,127280914.729,9939107.219
95 2025-04-18T06:33:35.000,609,23847253.773,23847269.953,125318282.949,97655888.826
96 2025-04-18T06:33:35.000,601,20954401.148,20954406.922,112014524.088,112014339.604
97 2025-04-18T06:33:35.000,602,21869913.367,21869921.438,21869921.438,21869921.355
98 2025-04-18T06:33:35.000,618,21683956.477,21683955.637,21683956.477,21683956.25
99 2025-04-18T06:33:35.000,615,2331429.306,2331429.891,21864839.832,21864839.844
100 2025-04-18T06:33:35.000,616,21473256.5,21473255.235,21473262.586,21473262.727
101 2025-04-18T06:33:35.000,617,19678444.396,19678444.406,19678444.396,19678444.396
102 2025-04-18T06:33:35.000,624,21563953.391,21563951.75,21563966.344,21563959.586
103 2025-04-18T06:33:35.000,613,26174791.805,26174802.137,26174799.934,26174802.621
104 2025-04-18T06:33:35.000,615,23483862.75,23483871.816,23483869.694,23483871.504
105 2025-04-18T06:33:35.000,621,26432398.532,26432398.934,26432398.532,26432398.532
106 2025-04-18T06:33:35.000,627,24194262.195,24194270.289,24194268.133,24194270.429
107 2025-04-18T06:33:35.000,630,25912772.523,25912783.785,25912781.77,25912783.895
108 2025-04-18T06:33:35.000,605,25488274.055,25488287.641,25488284.469,25488286.777
109 2025-04-18T06:33:35.000,613,38985382.773,38985312.443,25919121.391,156455528.146
110 2025-04-18T06:33:35.000,604,25023315.549,25023315.549,25023315.549,25023315.549
111 2025-04-18T06:33:35.000,603,37480154.182,37480154.551,18478452.159,150810583.123
112 2025-04-18T06:33:35.000,602,38895849.508,38895845.542,202489494.391,156577554.786
113 2025-04-18T06:33:35.000,601,37968455.211,37968456.809,197670850.346,152851024.181
114 2025-04-18T06:33:35.000,611,23461792.648,23461801.898,123292556.888,96072155.663
115 2025-04-18T06:33:35.000,617,28621716.75,28621722.957,108367848.235,84422592.274
116 2025-04-18T06:33:35.000,622,21902216.758,21902224.87,115804813.771,69077797.794
117 2025-04-18T06:33:35.000,609,2235925.195,2235924.566,116463564.832,90880537.346
118 2025-04-18T06:33:35.000,606,21087237.133,21087244.637,110814179.213,86348752.148
119 2025-04-18T06:33:35.000,619,1984587.602,1984591.113,104294519.956,8126471.316
120 2025-04-18T06:33:35.000,604,23532356.109,23532348.418,123663269.128,96361863.949
121 2025-04-18T06:33:35.000,614,22771472.039,22771482.777,119648687.433,92543384.071
122 2025-04-18T06:33:35.000,612,24224837.445,24224847.328,127280914.729,9939107.219
123 2025-04-18T06:33:35.000,609,23847253.773,23847269.953,125318282.949,97655888.826
124 2025-04-18T06:33:35.000,601,20954401.148,20954406.922,112014524.088,112014339.604
125 2025-04-18T06:33:35.000,602,21869913.367,21869921.438,21869921.438,21869921.355
126 2025-04-18T06:33:35.000,618,21683956.477,21683955.637,21683956.477,21683956.25
127 2025-04-18T06:33:35.000,615,2331429.306,2331429.891,21864839.832,21864839.844
128 2025-04-18T06:33:35.000,616,21473256.5,21473255.235,21473262.586,21473262.727
129 2025-04-18T06:33:35.000,617,19678444.396,19678444.406,19678444.396,19678444.396
130 2025-04-18T06:33:35.000,624,21563953.391,21563951.75,21563966.344,21563959.586
131 2025-04-18T06:33:35.000,613,26174791.805,26174802.137,26174799.934,26174802.621
132 2025-04-18T06:33:35.000,615,23483862.75,23483871.816,23483869.694,23483871.504
133 2025-04-18T06:33:35.000,621,26432398.532,26432398.934,26432398.532,26432398.532
134 2025-04-18T06:33:35.000,627,24194262.195,24194270.289,24194268.133,24194270.429
135 2025-04-18T06:33:35.000,630,25912772.523,25912783.785,25912781.77,25912783.895
136 2025-04-18T06:33:35.000,605,25488274.055,25488287.641,25488284.469,25488286.777
137 2025-04-18T06:33:35.000,613,38985382.773,38985312.443,25919121.391,156455528.146
138 2025-04-18T06:33:35.000,604,25023315.549,25023315.549,25023315.549,25023315.549
139 2025-04-18T06:33:35.000,603,37480154.182,37480154.551,18478452.159,150810583.123
140 2025-04-18T06:33:35.000,602,38895849.508,38895845.542,202489494.391,156577554.786
141 2025-04-18T06:33:35.000,601,37968455.211,37968456.809,197670850.346,152851024.181
142 2025-04-18T06:33:35.000,611,23461792.648,23461801.898,123292556.888,96072155.663
143 2025-04-18T06:33:35.000,617,28621716.75,28621722.957,108367848.235,84422592.274
144 2025-04-18T06:33:35.000,622,21902216.758,21902224.87,115804813.771,69077797.794
145 2025-04-18T06:33:35.000,609,2235925.195,2235924.566,116463564.832,90880537.346
146 2025-04-18T06:33:35.00
```

新建文本文档 (4) 与 (5) 的基线解算结果:

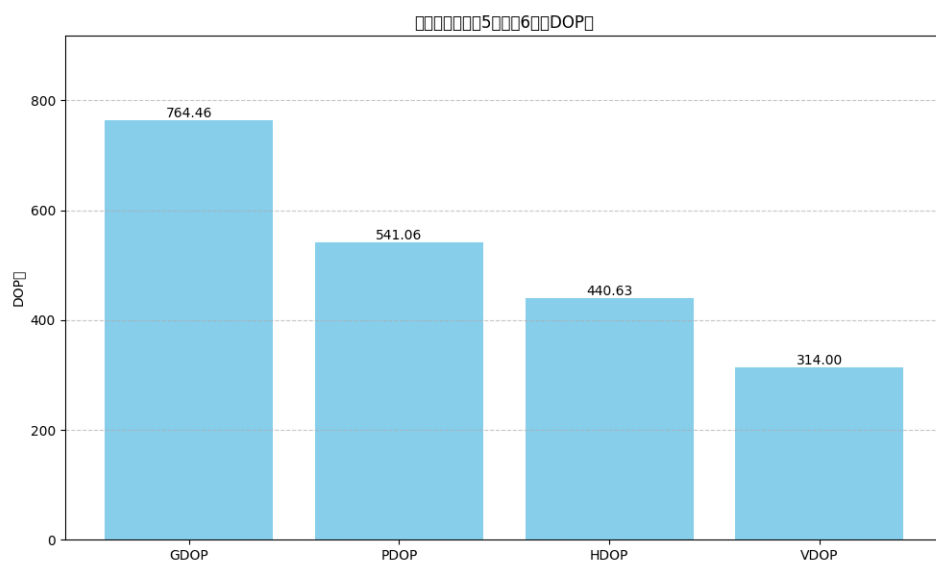
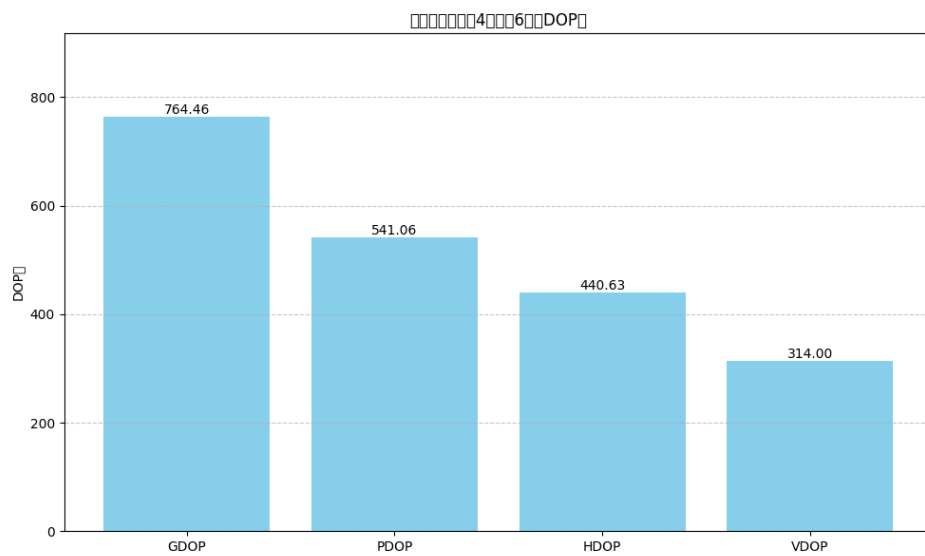
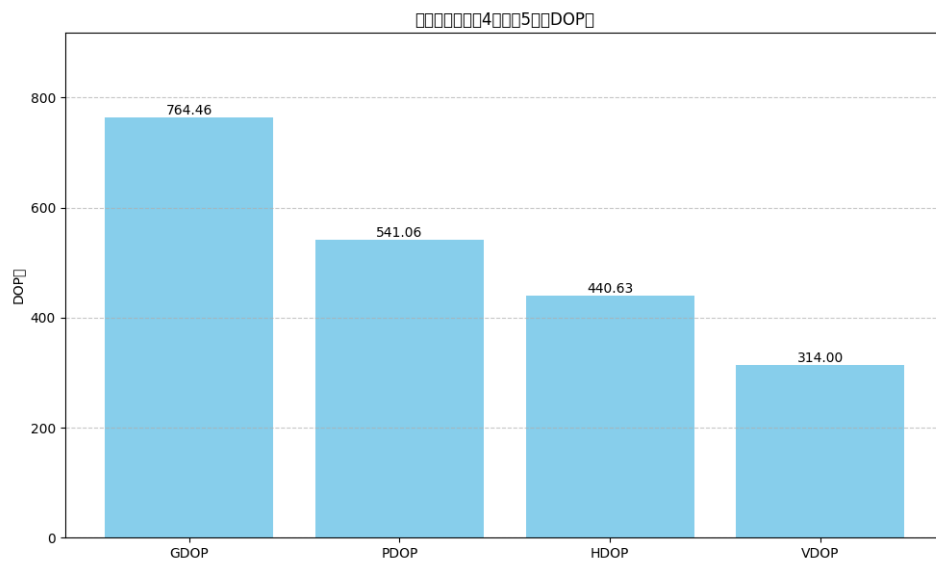
Satellite G20 Rover Position (X, Y, Z): -2616788.785760, 3981173.382290, 4233008.905945
Satellite G04 Rover Position (X, Y, Z): -2625830.275854, 3960654.918980, 4220179.497096
Satellite G12 Rover Position (X, Y, Z): -2615170.575352, 3978055.447592, 4233036.607930
Satellite G22 Rover Position (X, Y, Z): -2613343.412045, 3978477.514759, 4213788.321694
Satellite R18 Rover Position (X, Y, Z): -2621085.101629, 3981155.490765, 4227748.508363
Satellite C04 Rover Position (X, Y, Z): -2627219.528343, 3992525.219860, 4218959.126692
Satellite E27 Rover Position (X, Y, Z): -2617693.032529, 3956997.599858, 4220110.935893
Satellite E30 Rover Position (X, Y, Z): -2608360.872754, 3980205.078212, 4246252.237690
Satellite C02 Rover Position (X, Y, Z): -2635676.630805, 3974016.543260, 4218444.236192
Satellite E03 Rover Position (X, Y, Z): -2630092.745771, 3978287.109319, 4235875.535423
Satellite G11 Rover Position (X, Y, Z): -2627087.241619, 3975243.179870, 4222202.088671
Satellite R03 Rover Position (X, Y, Z): -2630953.790335, 3971114.135998, 4241857.908062
Satellite G19 Rover Position (X, Y, Z): -2614819.287457, 3971844.211775, 4221939.979722
Satellite C13 Rover Position (X, Y, Z): -2625292.679925, 3983577.328245, 4229201.783416
Satellite G09 Rover Position (X, Y, Z): -2609661.604885, 3977878.510887, 4224919.826862
Satellite R02 Rover Position (X, Y, Z): -2621711.703075, 3986885.294756, 4239735.475041
Satellite R01 Rover Position (X, Y, Z): -2631390.613256, 3980489.397846, 4212719.621589
Satellite R16 Rover Position (X, Y, Z): -2624287.937907, 3964763.992735, 4219229.241376
Satellite E13 Rover Position (X, Y, Z): -2625645.028808, 3973235.087708, 4236365.236830
Satellite E15 Rover Position (X, Y, Z): -2631227.656983, 3970900.696182, 4235010.733033
Satellite G06 Rover Position (X, Y, Z): -2610026.416001, 3977998.556119, 4233810.377741
Satellite C03 Rover Position (X, Y, Z): -2622544.389150, 3979341.394032, 4214757.348708
Satellite R17 Rover Position (X, Y, Z): -2615724.510069, 3974793.614420, 4222822.876307
Satellite E05 Rover Position (X, Y, Z): -2626918.863522, 3983843.927345, 4234419.880388
Satellite R24 Rover Position (X, Y, Z): -2612479.951916, 3988158.711843, 4237842.004973
Satellite G17 Rover Position (X, Y, Z): -2619252.143452, 3993004.428889, 4223700.306181
Satellite E21 Rover Position (X, Y, Z): -2619469.516899, 3964153.517335, 4221980.208951
Satellite R19 Rover Position (X, Y, Z): -2618115.931206, 3981550.875877, 4241724.359724
Satellite R09 Rover Position (X, Y, Z): -2628088.111558, 3976898.383680, 4231702.982709

新建文本文档 (4) 与 (6) 的基线解算结果:

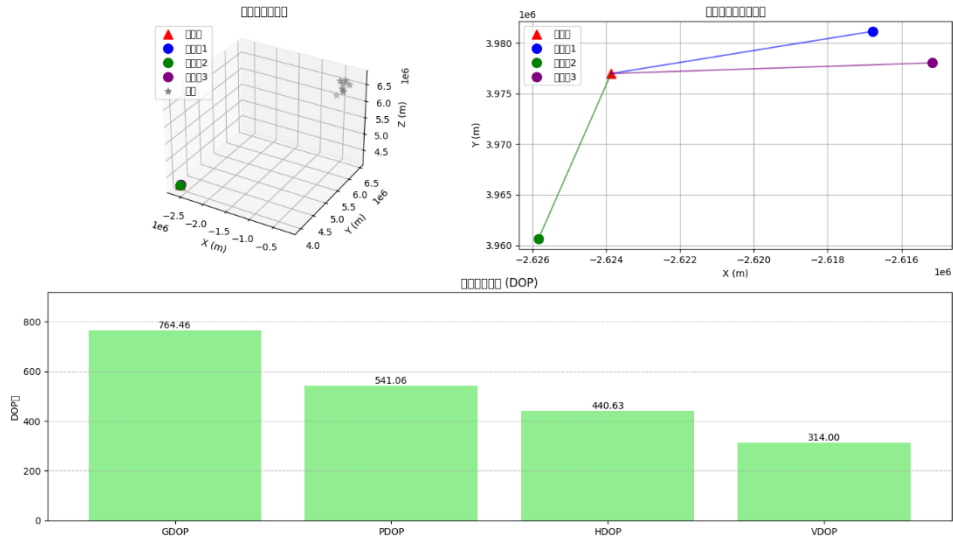
Satellite G20 Rover Position (X, Y, Z): -2625116.850716, 3974144.960886, 4241134.727492
Satellite G03 Rover Position (X, Y, Z): -2617607.372018, 3981373.860680, 4227754.630645
Satellite G04 Rover Position (X, Y, Z): -2616984.786632, 3976911.648825, 4234958.400727
Satellite G12 Rover Position (X, Y, Z): -2623883.302926, 3973020.200917, 4226672.998367
Satellite G22 Rover Position (X, Y, Z): -2630721.968903, 3992882.951746, 4224205.928558
Satellite R18 Rover Position (X, Y, Z): -2632152.605789, 3980822.530419, 4232255.996969
Satellite C04 Rover Position (X, Y, Z): -2623071.987826, 3974623.118019, 4220857.977026
Satellite E27 Rover Position (X, Y, Z): -2630024.690861, 3978109.698781, 4227651.092862
Satellite E30 Rover Position (X, Y, Z): -2613642.291440, 3975768.011655, 4227786.906139
Satellite C02 Rover Position (X, Y, Z): -2621216.204135, 3980341.043642, 4218664.083528
Satellite E03 Rover Position (X, Y, Z): -2625953.691634, 3972859.973598, 4239202.622950
Satellite G11 Rover Position (X, Y, Z): -2620877.903722, 3978661.198465, 4223202.675674
Satellite R03 Rover Position (X, Y, Z): -2617536.653532, 3970812.716073, 4232965.436658
Satellite G19 Rover Position (X, Y, Z): -2621835.044619, 3972762.152779, 4227723.084489
Satellite C13 Rover Position (X, Y, Z): -2624276.808827, 3972749.427334, 4224776.787225
Satellite G09 Rover Position (X, Y, Z): -2620609.682198, 3973795.665452, 4231936.830885
Satellite R02 Rover Position (X, Y, Z): -2625249.627020, 3981267.113873, 4230606.518632
Satellite R01 Rover Position (X, Y, Z): -2623393.082649, 3974739.576437, 4225211.404022
Satellite E13 Rover Position (X, Y, Z): -2632976.651407, 3976594.021044, 4221761.646021
Satellite R16 Rover Position (X, Y, Z): -2625732.936471, 3980082.118241, 4227080.710270
Satellite E15 Rover Position (X, Y, Z): -2633118.253174, 3964039.902977, 4218165.651246
Satellite G06 Rover Position (X, Y, Z): -2625909.827650, 3980840.036828, 4228879.610443
Satellite C03 Rover Position (X, Y, Z): -2629283.956631, 3969253.368603, 4219125.087021
Satellite R17 Rover Position (X, Y, Z): -2628730.213959, 3977152.990690, 4240257.358623
Satellite E05 Rover Position (X, Y, Z): -2624219.095711, 3982711.252112, 4223343.434594
Satellite R24 Rover Position (X, Y, Z): -2620845.728360, 3958661.508495, 4217669.205560
Satellite G17 Rover Position (X, Y, Z): -2622261.871546, 3971922.532197, 4229865.916259
Satellite E21 Rover Position (X, Y, Z): -2626110.127936, 3976013.623938, 4231872.865005
Satellite C01 Rover Position (X, Y, Z): -2623831.376743, 3977046.042643, 4226137.278394
Satellite R09 Rover Position (X, Y, Z): -2623894.456918, 3977049.427153, 4226180.932205
Satellite R19 Rover Position (X, Y, Z): -2623881.091283, 3977096.584128, 4226072.902344

新建文本文档 (5) 与 (6) 的基线解算结果:

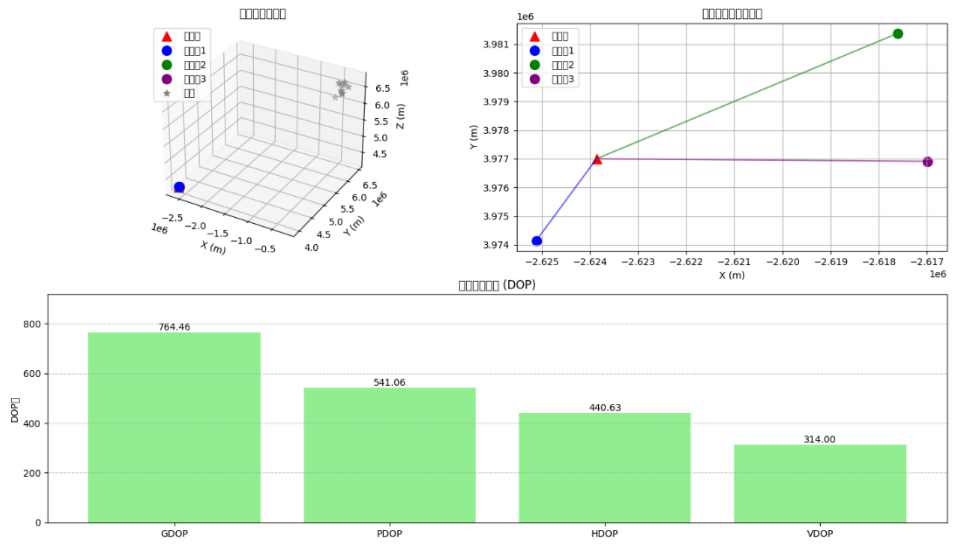
Satellite G20 Rover Position (X, Y, Z): -2627274.994907, 3971799.645792, 4230856.845277
Satellite G04 Rover Position (X, Y, Z): -2627420.444693, 3975120.431391, 4216837.574512
Satellite G12 Rover Position (X, Y, Z): -2632171.603625, 3980299.405671, 4229440.298754
Satellite G22 Rover Position (X, Y, Z): -2622638.150782, 3974669.379561, 4234522.949139
Satellite R18 Rover Position (X, Y, Z): -2627282.306207, 3980644.941562, 4217151.916111
Satellite C04 Rover Position (X, Y, Z): -2616966.397379, 3975090.825966, 4217844.857532
Satellite E27 Rover Position (X, Y, Z): -2623152.682235, 3972456.174090, 4217248.367287
Satellite C02 Rover Position (X, Y, Z): -2609045.655491, 3967297.927217, 4231613.860438
Satellite E03 Rover Position (X, Y, Z): -2625507.996507, 3972487.825368, 4223039.097506
Satellite G11 Rover Position (X, Y, Z): -2628504.660727, 3978350.266294, 4219292.632354
Satellite R03 Rover Position (X, Y, Z): -2625524.509244, 3971175.978922, 4224127.453180
Satellite G19 Rover Position (X, Y, Z): -2627145.036639, 3980588.703445, 4221673.235185
Satellite C13 Rover Position (X, Y, Z): -2613119.162467, 3970622.068484, 4222074.356881
Satellite G09 Rover Position (X, Y, Z): -2625999.353609, 3990102.508254, 4221328.414855
Satellite R02 Rover Position (X, Y, Z): -2622728.734424, 3977937.430160, 4227677.735984
Satellite R01 Rover Position (X, Y, Z): -2624655.344260, 3977285.103338, 4229115.525519
Satellite R09 Rover Position (X, Y, Z): -2621828.902452, 3975314.217971, 4224488.546841
Satellite E13 Rover Position (X, Y, Z): -2617706.649700, 3978885.345085, 4216902.451016
Satellite R16 Rover Position (X, Y, Z): -2614958.968153, 3982371.472633, 4220366.269277
Satellite E15 Rover Position (X, Y, Z): -2620135.152008, 3967328.746043, 4230997.248262
Satellite G06 Rover Position (X, Y, Z): -2623989.506041, 3982604.636728, 4226685.118160
Satellite C03 Rover Position (X, Y, Z): -2621782.791624, 3979626.560056, 4231683.191305
Satellite R17 Rover Position (X, Y, Z): -2623851.758050, 3978644.874099, 4228522.517146
Satellite E05 Rover Position (X, Y, Z): -2617722.190618, 3979536.907360, 4235422.221610
Satellite R24 Rover Position (X, Y, Z): -2620941.672879, 3978099.844024, 4230260.794936
Satellite G17 Rover Position (X, Y, Z): -2623333.120891, 3961078.726460, 4230943.956260
Satellite E21 Rover Position (X, Y, Z): -2619775.420725, 3966312.889249, 4225258.755079
Satellite E30 Rover Position (X, Y, Z): -2625431.588745, 3976285.984880, 4225271.113549
Satellite R19 Rover Position (X, Y, Z): -2627731.650246, 3980905.113895, 4232348.289867



00000004000500000000



□□□□□□4□□□6□□□□□□□□



□□□□□□5□□□6□□□□□□□□

